02. Python Syntax & Comments

Created by Li Huanyu, last modified by suncaper on Mar 09, 2026

Introduction

Syntax is the set of rules that defines how a Python program is written and interpreted. If you break these rules, Python will stop and give you a `SyntaxError`.

In this lesson, we will cover the two most fundamental rules of Python:

1. **Indentation** (How code is grouped).
2. **Comments** (How code is explained).

Part 1: Indentation (The Golden Rule)

In many programming languages, indentation is just for aesthetics. In Python, **indentation is mandatory**. It tells Python which lines of code belong together.

1. How it works

- A "block" of code (like inside a loop or a function) starts with a colon (`:`) and is indented.
- The standard indentation is **4 spaces** (or 1 Tab).

2. Correct Example ✅

```
if 5 > 2:
    print("Five is greater than two!")  # This line is indented
    print("This is also inside the if-block.")
```

3. Incorrect Example ❌

If you skip the space, Python gets confused.

```
if 5 > 2:
print("Five is greater than two!")  # <--- IndentationError: expected an indented block
```

Python vs. The "Curly Brace" Languages
Most major programming languages (Java, C++, JavaScript, and **Scala**) belong to the "C-Style" family. They use **Curly Braces {}** to group code and **Semicolons ;** to end lines.

Python removes all of that noise.

1) Java / Scala Example (The "Hard" Way)

In the Big Data world, **Scala** is the native language of Apache Spark. If you look at Spark documentation, you might see code like this:

```
// Java or Scala Syntax
if (userAge > 18) {
    // The brace '{' starts the block
    System.out.println("User is an adult");
    saveToDatabase();
} // The brace '}' ends the block
```

- **Visual Noise:** You must type (,), {, }, and ;.
- **Indentation:** In Java/Scala, indentation is optional. The computer doesn't care if the code is messy, as long as the braces {} are there.

2) Python Example (The "Clean" Way)

Python replaces the braces with **Colons (:)** and **Indentation**.

```
# Python Syntax
if user_age > 18:
    # The colon ':' starts the block
    # The indentation (4 spaces) defines the block
    print("User is an adult")
    save_to_database()
# The block ends when we stop indenting
```

3) Side-by-Side Comparison

Let's look at a "Loop" (doing something repeatedly).

Feature	Java / Scala (Legacy Big Data)	Python (Modern Data Engineering)
Start Block	for (i=0; i<5; i++) {	for i in range(5):
Inside Block	System.out.println(i);	print(i)
End Block	}	(Just stop indenting)
End of Line	; (Semicolon required)	(Newline is enough)

Why does this matter?

1. **Readability:** Python forces you to write neat code. You cannot write messy, unindented code in Python—it simply won't run.
2. **Speed:** You type fewer characters. In the long run, Python code is often 3-5x shorter than Java code for the same task.
3. **Spark Context:** When you move to **PySpark**, you will realize that writing data transformations in Python feels much more like writing English instructions compared to the strict syntax of Scala or Java.

4. The "Big Data" Context

Why does this matter? When you write PySpark code later, you will often nest logic inside logic. For example:

> *"For every user (Level 1), check if they bought an item (Level 2), and if the price is > $100 (Level 3)."*

Your code will look like a staircase. If the indentation is wrong, the logic breaks.

Part 2: Comments (Documentation)

Comments are lines of text that Python ignores. They are written for **humans** (you and your teammates), not the computer.

1. Single-Line Comments (#)

Use the hash symbol `#`. Everything after it is ignored.

```
# This is a comment. Python will not run this.
print("Hello, World!") # You can also put comments at the end of a line.
```

2. Multi-Line Comments (Docstrings)

Technically, Python does not have a specific "multi-line comment" syntax, but we use **triple quotes** (`"""`) to create multi-line text strings that act as comments.

```
"""
This is a multi-line comment.
We can write as many lines as we want.
This is often used to describe what a complex script does.
"""
print("The comment above is ignored.")
```

3. Best Practices for Big Data

In Data Engineering, code can get very complex. You should use comments to explain **WHY**, not **WHAT**.

- **Bad Comment:**

  ```
  x = x + 1  # Add 1 to x

  (We can see that. It's obvious.)
  ```

- **Good Comment:**

  ```

  ```

  (This explains the logic behind the code.)

Part 3: Python Case Sensitivity

Python is **case-sensitive**. This means `Age`, `age`, and `AGE` are three different things.

```
# These are considered two different variables
my_variable = 10
My_Variable = 20

print(my_variable) # Prints 10
print(My_Variable) # Prints 20
```

Common Mistake: Typing `Print()` instead of `print()`.

```
Print("Hello") # <--- NameError: name 'Print' is not defined
```

Part 4: Hands-on Exercise

Let's write a small script that combines syntax, indentation, and comments.

1. Open PyCharm.
2. Create a new file named `syntax_practice.py`.
3. Type the following code exactly as shown (pay attention to the 4 spaces!):

```
# ----------------------------------------------------------
# Big Data Simulation: Simple Data Filter
# ----------------------------------------------------------

"""
This script simulates checking a user's age
to see if they are eligible for a data survey.
"""

user_age = 25
user_name = "Alice"

# Logic to check eligibility
if user_age > 18:
    # Everything inside here runs ONLY if age > 18
    print("Checking database for:", user_name)
    print("Status: User is an adult.")

    if user_age > 60:
        # This is a nested block (8 spaces deep)
        print("Category: Senior Citizen")
    else:
        print("Category: Adult")

# This line is NOT indented, so it runs no matter what
print("--- End of Check ---")
```

Run the code.

```
Checking database for: Alice
Status: User is an adult.
Category: Adult
--- End of Check ---
```

What happens?

- Python reads top to bottom.
- It ignores the comments.
- It sees `user_age` is 25.
- It enters the first `if` block (because 25 > 18).
- It skips the nested `if user_age > 60` block (because 25 is not > 60).
- It runs the `else` block.
- Finally, it prints "--- End of Check ---".

Summary

1. **Indentation** is structural. Use 4 spaces.
2. **Comments** start with `#`. Use them to explain *why* you wrote the code.
3. **Case Sensitivity** matters (`print` vs `Print`).

No labels